

Cross-Chain Attestation Portability

Stefan Stefanović, Ligate Labs

2026-05-26

Cross-Chain Attestation Portability

Unified Mechanism for IBC, Hyperlane, and Restaking-Style Bridging

Ligate Labs Research, Working Paper v0.2

Date: 2026-05-27

Status: v0.2 promotes the v0.1 outline to substantive content across all sections. §3 specifies the unified IBC-style light-client proof primitive with freshness commitment and revocation-status query. §4 walks each of the five per-extension mechanisms with concrete adaptations. §5 quantifies the update-latency vs revocation-latency trade-off per extension. §6 specifies cross-chain slashing-event validation with double-slash prevention. The paper stays target-agnostic at v0.2 (IBC vs Hyperlane vs restaking-style); the primitive is specified in IBC vocabulary because IBC is the most mature, but the mechanism generalizes. v0.3 commits to a specific bridging target once the Ligate Labs engineering decision lands.

Contact: hello@ligate.io

Version history: v0.1 (2026-05-26, outline). v0.2 (2026-05-27, substantive content across all sections + per-extension mechanism + latency analysis + slashing-event validation).

Contents

Cross-Chain Attestation Portability	1
Unified Mechanism for IBC, Hyperlane, and Restaking-Style Bridging	1
1. Introduction	3
1.1 Why one paper for five extensions	3
1.2 The chain-on-Celestia rollup-portability constraint	3
1.3 The IBC-vs-Hyperlane-vs-restaking choice	3
1.4 The central question	4
1.5 Approach in brief	4
1.6 Contributions	4
1.7 Scope and non-goals	4
1.8 Document structure	5
2. Background and Related Work	5
2.1 IBC (Inter-Blockchain Communication)	5
2.2 Hyperlane	5
2.3 Wormhole, Polymer, LayerZero	5
2.4 Restaking-style attestation portability (EigenLayer-adjacent)	5
2.5 Ethereum cross-validator portability (EIP-7002 / EIP-7251)	5
3. The Unified IBC Light-Client Proof Primitive	5
3.1 Proof structure	5
3.2 Freshness commitment	6
3.3 Revocation-status query	7
3.4 Cascade adaptation	7
4. Per-Extension Mechanisms	8
4.1 Cross-chain delegation (native-delegation v0.2 §10.2)	8
4.2 Cross-chain fee-market portability (per-schema-fees v0.2 §9.4)	8
4.3 Cross-chain composition (cross-schema-composition v0.2 §9.1)	8
4.4 Cross-chain time-locks (time-locked-attestations v0.2 §9.2)	8
4.5 Cross-chain DA-layer queries (native-da v0.2 §13.4)	8
5. Update-Latency vs Revocation-Latency Analysis	8
5.1 Bounded-stale window	9
5.2 Per-extension damage bounds	9
5.3 Mitigations	9
6. Cross-Chain Slashing-Event Validation	10
6.1 Slashing events reported to Ligate via IBC	10
6.2 Slashing events reported by Ligate to counterparties	10
6.3 Race conditions and double-slash prevention	10
7. Roadmap	10
7.1 v0: no cross-chain	10
7.2 v1.5: first extension shipped	10
7.3 v2: multiple extensions	10
7.4 v3+: full surface	10
8. Conclusion	11
References	12
Appendix A: Per-extension worked example	12
Appendix B: IBC packet format spec	12

Five existing v0.2 papers each defer a cross-chain extension to a unified follow-up: native-delegation v0.2 §10.2 (cross-chain delegation), per-schema-fees v0.2 §9.4 (cross-chain fee-market portability), cross-schema-composition v0.2 §9.1 (cross-chain composition), time-locked-attestations v0.2 §9.2 (cross-chain time-locks), and native-da v0.2 §13.4 (cross-chain DA-layer queries). This paper consolidates the five extensions into one coherent design built on a single primitive.

The primitive is an **IBC-style light-client proof of Ligate-side state that counterparty chains can verify locally**, plus a freshness commitment that bounds the staleness of the proof. The same primitive (in reverse) lets Ligate-side admission checks verify counterparty-chain state. The paper specifies the proof primitive in §3, the per-extension mechanism in §4 (five subsections, one per upstream paper), the update-latency vs revocation-latency trade-off in §5, and cross-chain slashing-event validation in §6.

The framing throughout: one shared mechanism rather than five overlapping ad hoc designs. Specifying the cross-chain extensions in five separate papers would re-derive the same primitives five times; specifying them in one paper provides a single reference for chain implementers, security reviewers, and integration partners.

The paper stays target-agnostic at v0.2. The primitive is specified in IBC vocabulary because IBC is the most mature trust-minimized bridge protocol, but the mechanism generalizes to Hyperlane (interoperator-validator-based trust) and restaking-style (Ethereum-rooted trust) targets. The engineering choice of target is downstream of this paper; v0.3 commits once that choice is made.

1. Introduction

1.1 Why one paper for five extensions

Each of the five upstream papers identifies a cross-chain extension as out of scope; each notes the same shape of mechanism (IBC light-client proof + freshness commitment + cascade adaptation). Specifying these in five separate papers would re-derive the same primitives five times. This paper specifies the shared mechanism once and applies it per extension in §5.

1.2 The chain-on-Celestia rollup-portability constraint

Ligate Chain runs as a Sovereign SDK rollup on Celestia. IBC-aware Cosmos chains expect IBC-native light-client proofs; Sovereign SDK rollup state proofs are not native IBC primitives in most Cosmos chains today. The paper documents this engineering gap and specifies the bridge primitive that closes it; the actual chain-side implementation is engineering work tracked separately.

1.3 The IBC-vs-Hyperlane-vs-restaking choice

Three target options. (a) Pure IBC: maximum interoperability with Cosmos ecosystem; requires Ligate light-client portability. (b) Hyperlane: interoperator-validator-based message passing; easier to deploy initially, weaker security than IBC. (c) Restaking-style (EigenLayer-adjacent): use Ethereum L1 as the trust root for cross-chain Ligate messages. v0.2 substantive content commits to one target; v0.1 stays target-agnostic and specifies the primitive in IBC’s vocabulary.

1.4 The central question

Given an attestation on Ligate Chain, what is the minimum chain-side primitive that lets an IBC-connected counterparty chain (or a Hyperlane-bridged chain, or a restaking-style verifier) verify the attestation’s existence, current validity, and (when applicable) the cascade-status of its dependencies, without trusting an off-chain coordinator?

The paper’s answer is the §3 IBC-style light-client proof primitive with freshness commitment, plus the §3.3 revocation-status query for explicit current-validity checks, plus the §3.4 cascade-visibility-latency bound for dependency tracking. The primitive is target-agnostic (works under IBC, Hyperlane, or restaking-style trust roots) and per-extension applications follow in §4.

1.5 Approach in brief

§3 surveys related work. §4 specifies the unified IBC light-client proof primitive. §5 walks the five upstream papers’ extensions and shows how each uses the primitive. §6 analyzes the update-latency vs revocation-latency trade-off per extension. §7 specifies cross-chain slashing-event validation. §8 lays out the roadmap (v0 none, v1.5 one extension, v2 multiple, v3+ full).

1.6 Contributions

1. **Unified mechanism.** One primitive (IBC light-client proof + freshness commitment) handles all five upstream papers’ cross-chain extensions.
2. **Per-extension mechanism specification.** Five subsections, one per upstream paper, showing how the unified primitive is consumed.
3. **Latency analysis.** Bounded damage per extension under realistic IBC round-trip latency.
4. **Cross-chain slashing-event validation.** PoUA reputation on Ligate accepting a slashing event reported via IBC; counterparty chains validating Ligate-side slashes against their own slashing modules.
5. **Roadmap.** Phased rollout from no cross-chain (v0) through full surface (v3+).

1.7 Scope and non-goals

In scope:

- Unified primitive specification (IBC light-client proof + freshness)
- Per-extension mechanism for each of the five upstream papers
- Update-latency vs revocation-latency trade-off
- Cross-chain slashing-event validation
- Roadmap aligned with chain-side engineering pace

Explicitly out of scope:

- Picking the engineering target (IBC vs Hyperlane vs restaking); this is an engineering decision downstream of the paper
- Specifying the Sovereign SDK rollup-state-proof bridge (engineering gap, documented but not closed by this paper)
- EVM-compatible bridge wrappers (out of scope; chain primitives only)
- Cross-chain fiat / off-ramp mechanics
- Bridge-protocol endorsement (the paper is neutral on partner choice)

1.8 Document structure

§2 surveys IBC / Hyperlane / Wormhole / Polymer / restaking-style alternatives. §3 specifies the unified IBC light-client proof primitive. §4 walks the five per-extension mechanisms. §5 covers latency analysis. §6 covers cross-chain slashing. §7 lays out the roadmap. §8 concludes.

2. Background and Related Work

Survey of bridge / portability designs in the broader ecosystem.

2.1 IBC (Inter-Blockchain Communication)

Cosmos SDK's IBC. Light-client-based, trust-minimized, mature. Standard model for Cosmos-to-Cosmos chain communication. Limitation: requires light-client portability of the chain's state proofs, which Sovereign SDK rollups do not currently have as a native IBC primitive.

2.2 Hyperlane

Interoperator-validator-based message passing. Easier to deploy than IBC; trust model is “Hyperlane operator validator set is honest” rather than “the chain's light client is correct.” Weaker security but easier interoperability across non-Cosmos chains.

2.3 Wormhole, Polymer, LayerZero

Compare and contrast briefly. Trust models, latency profiles, ecosystem adoption.

2.4 Restaking-style attestation portability (EigenLayer-adjacent)

Use Ethereum L1 (or another base layer) as the trust root for cross-chain Ligate messages. Validators stake on Ethereum; their stake is slashable for cross-chain message correctness. Different security model: trust the restaking-validator set rather than the chain's light client.

2.5 Ethereum cross-validator portability (EIP-7002 / EIP-7251)

Ethereum's own approach to cross-validator portability via EIP-7002 (triggerable exits) and EIP-7251 (max effective balance). Reference for what the broader ecosystem considers state-of-the-art for cross-chain validator messaging.

3. The Unified IBC Light-Client Proof Primitive

The single mechanism that all five upstream papers' cross-chain extensions consume.

3.1 Proof structure

An IBC packet carrying Ligate-side state to a counterparty chain has four components:

Component	Purpose	Source
<code>attestation_object</code>	The attestation data the counterparty needs	Ligate state tree at block height h
<code>inclusion_proof</code>	Merkle proof of inclusion in the state tree	Computed from Ligate state tree
<code>header_h</code>	Ligate-side block header at height h , signed by attester set quorum	Ligate consensus output
<code>freshness_commitment</code>	Counterparty’s most recent known finalized Ligate header	Counterparty light client state

The counterparty light client verifies each component locally. (a) Attestation object is parsed against the relevant schema; (b) inclusion proof verifies the attestation is in the state tree at height h ; (c) header signature verifies against the counterparty’s known attester-set roster; (d) freshness commitment requires `h >= freshness.height`. If all four verify, the counterparty accepts the attestation as authoritative for its purposes.

Trust assumption: the counterparty’s light client is correctly tracking Ligate’s attester-set turnover. Under standard IBC, this is the IBC client update mechanism; under Hyperlane, this is the inter-operator validator set; under restaking, this is the Ethereum-rooted slashing protocol. The §6 slashing-event validation specifies what happens if Ligate-side slashing must propagate to the counterparty.

3.2 Freshness commitment

The freshness commitment is what makes the primitive bounded-stale. Without it, an attacker could replay an old IBC packet showing an attestation that has since been revoked: the attestation existed at the time of the snapshot, but no longer holds the property the consumer relies on.

The freshness commitment names the most recent Ligate header the counterparty has seen. Each counterparty-side light client tracks `(latest_ligate_height, latest_ligate_header_hash)`. When an IBC packet arrives carrying an attestation proof at height h , the counterparty checks: $h \geq \text{latest_ligate_height} - W$ for a window W governance-set by the counterparty (typical $W = 1\text{h}$ of blocks, or roughly 300 Ligate blocks at 12-second block time).

The counterparty advances `latest_ligate_height` as new IBC headers come in. Under standard IBC, header updates happen every IBC update cycle (typically 30 seconds to several minutes depending on relayer cadence). Under Hyperlane, every interoperator-validator-set message. Under restaking, every Ethereum-rooted attestation. The freshness window W trades update-frequency against stale-attestation acceptance: a tighter W rejects more stale proofs but requires more frequent IBC updates.

For high-stakes cross-chain compositions (cross-chain attester mints, cross-chain delegation), recommend $W = 30\text{ min}$ (150 Ligate blocks). For low-stakes (cross-chain content credentials), $W = 1\text{ h}$ is adequate.

3.3 Revocation-status query

Beyond the attestation’s existence at height h , the counterparty often needs the attestation’s *current* status: valid, revoked, or cascade-invalidated (per CSC v0.2 §5). The IBC primitive supports a “current-state-of-attestation” query operation:

1. Counterparty sends an IBC packet to Ligate carrying `query(attestation_uid)`.
2. Ligate’s IBC handler reads the current state of the attestation: `status = {VALID, REVOKED, CASCADE_INVALIDATED}` plus the timestamp of the most recent status change.
3. Ligate returns a signed response with the status and the height at which the response was generated.
4. Counterparty verifies the signature against its known Ligate attester-set roster and accepts the status.

Latency: one IBC round-trip per query. For composition workflows where revocation visibility matters (the counterparty’s downstream attestation should cascade if the Ligate input is revoked), the counterparty queries before admitting the dependent attestation. For workflows that accept bounded staleness (most low-stakes cross-chain consumers), the counterparty relies on freshness-bound updates instead of explicit queries.

The query primitive is essentially the same IBC mechanism in §3.1 in reverse: a query packet from counterparty to Ligate, a response packet from Ligate back to counterparty, each verified against the other’s light client.

3.4 Cascade adaptation

CSC v0.2 §5 BFS-cascade fires on Ligate when an upstream attestation is revoked: all dependent Ligate-side attestations are marked invalid via traversal of the dependency graph. The cascade is bounded in depth and gas-charged to the revocation root.

Dependent attestations on counterparty chains do not auto-update when the Ligate-side cascade fires. The counterparty learns of the cascade in one of three ways:

- (a) **IBC update**: when the counterparty’s light client advances its `latest_ligate_height`, the next admission check or revocation-status query sees the updated state. Latency: bounded by IBC update cadence (typically 30 seconds).
- (b) **Explicit query**: the counterparty queries Ligate via §3.3 for an attestation’s status before consuming it downstream. Latency: one IBC round-trip per query.
- (c) **Push notification (v0.3+ extension)**: Ligate can push status changes to counterparty chains that have subscribed to the dependency root. This would require an additional IBC packet type and counterparty-side subscription state. Out of scope for v0.2; flagged as a potential v0.3 extension.

For v0.2, the counterparty’s cascade-visibility latency is bounded by `IBC update cadence + header-finality time`. Workflows that need tighter cascade visibility use §3.3 explicit queries; workflows that accept bounded staleness rely on IBC updates.

4. Per-Extension Mechanisms

How each of the five upstream papers' cross-chain extensions uses the §3 primitive.

4.1 Cross-chain delegation (native-delegation v0.2 §10.2)

A transaction on counterparty chain C wants to be authorized by a delegation grant on Ligate. The counterparty receives an IBC packet carrying (the grant, its inclusion proof at height h , header h signed). Verifies the proof; admits the transaction. Revocation: counterparty's view of the grant is bounded-stale by one IBC update. Trade-off documented in §5.

4.2 Cross-chain fee-market portability (per-schema-fees v0.2 §9.4)

A schema registered on Ligate is also recognized on counterparty C for fee-market purposes. C needs the current base fee b_σ to price its attestations of the same schema. The IBC primitive carries (b_σ, h) ; C 's fee-market reads this on each update. Bounded-stale by IBC update latency; counterparty's fee may slightly diverge from Ligate's during the window.

4.3 Cross-chain composition (cross-schema-composition v0.2 §9.1)

A CSC v0.2 §4.3 typed reference can target an attestation on a counterparty chain. The Ligate-side admission check uses §3.1 in reverse: the counterparty's attestation is presented to Ligate with a counterparty-light-client proof. The Ligate-side admission predicate verifies the cross-chain proof. Cascade semantics: if the cross-chain input is revoked, Ligate cascade fires after the next IBC update reveals the revocation.

4.4 Cross-chain time-locks (time-locked-attestations v0.2 §9.2)

A TLA on Ligate whose reveal is gated by counterparty block height. The Ligate-side admission predicate for the reveal accepts an IBC light-client proof of the counterparty's current height h_C . If $h_C \geq$ the time-lock target, the reveal admits. Latency: target height must be far enough in the future that IBC update latency is absorbed by the target's margin.

4.5 Cross-chain DA-layer queries (native-da v0.2 §13.4)

A counterparty chain wants to query an attestation by its native-DA chunk. The IBC primitive carries the DA chunk pointer; the counterparty's DA light client (or a wrapping bridge) fetches and verifies. Operational dependency: counterparty needs sampling access to Ligate's native DA. v0.2 substantive content specifies whether this requires light-client portability (likely yes) or a custom bridge protocol.

5. Update-Latency vs Revocation-Latency Analysis

All five extensions inherit the same shape of trade-off: the counterparty's view of Ligate state is bounded-stale by IBC update cadence. Different extensions have different damage profiles under staleness.

5.1 Bounded-stale window

With IBC updates every 30 seconds (typical Cosmos IBC), the counterparty’s view of Ligate state lags by 30 seconds in expectation. Revocation events on Ligate are visible to the counterparty after the next update.

5.2 Per-extension damage bounds

Per-extension damage analysis under realistic 30-second IBC update window:

Extension	Damage profile	Bound
§4.1 Cross-chain delegation	Up to one IBC window of transactions admitted on counterparty using a now-revoked grant.	Grant’s transaction-rate cap $\times$ IBC window (30s)
§4.2 Cross-chain fee-market	Bounded fee divergence between Ligate-side b_σ and counterparty’s snapshot. No security implications; price-precision loss only.	Per-block base-fee delta $\times$ window
§4.3 Cross-chain composition	Up to one window of dependent attestations admit referencing a now-cascade-invalidated input. Cascade fires retroactively when the counterparty’s update lands.	Per-block compose-rate $\times$ window
§4.4 Cross-chain time-locks	No damage. The reveal predicate uses the counterparty’s <i>current</i> height directly, which is naturally fresh on the counterparty side.	n/a
§4.5 Cross-chain DA queries	Bounded DA-fetch latency. No security implications if counterparty has light-client access to the DA layer.	DA round-trip latency

The §5.3 mitigations reduce damage where it matters (delegation and composition); the time-lock extension has no damage by construction.

5.3 Mitigations

Two paths to reduce damage. (a) Faster IBC update cadence (more frequent updates, higher gas cost for the relayer). (b) Pre-commitment patterns (Ligate-side actions signal pending revocation N blocks in advance, giving the counterparty time to drain). Both are governance-tunable and v0.2 specifies recommended values.

6. Cross-Chain Slashing-Event Validation

PoUA reputation on Ligate accepting a slashing event reported via IBC; counterparty chains validating Ligate-side slashes against their own slashing modules.

6.1 Slashing events reported to Ligate via IBC

If a Ligate-attestor-set member misbehaves on a counterparty chain (e.g., signs a conflicting attestation on a remote chain), the counterparty reports the misbehavior to Ligate via IBC. Ligate validates the proof, applies the slash via the `disputes` module, and updates PoUA reputation. Mechanism reuses the §3 primitive in reverse.

6.2 Slashing events reported by Ligate to counterparties

If a counterparty-side attestor (whose cross-chain authority depends on Ligate-side slashable stake) misbehaves on the counterparty, the counterparty's slashing module slashes the counterparty-side balance. Ligate's view of the attestor's reputation updates after the next IBC update.

6.3 Race conditions and double-slash prevention

If misbehavior is simultaneously reported on both chains, the slashing-module on each chain must coordinate to avoid double-slash beyond the §A.3 cap. v0.2 specifies the coordination protocol.

7. Roadmap

Phased rollout aligned with chain-side engineering.

7.1 v0: no cross-chain

Devnet operates without any cross-chain extension. All five upstream papers' extensions are out of scope at v0.

7.2 v1.5: first extension shipped

Ship the §3 primitive plus one extension. Recommendation: cross-chain delegation (§4.1) ships first because it has the clearest product use case (Iris-relayed agents acting across Cosmos chains).

7.3 v2: multiple extensions

Add §4.2 (cross-chain fee-market) and §4.3 (cross-chain composition). Engineering work compounds; mainnet hardening accumulates.

7.4 v3+: full surface

§4.4 (cross-chain time-locks) and §4.5 (cross-chain DA queries) ship as the cross-chain workload matures. Cross-chain slashing-event validation (§6) is co-shipped with the first extension that requires cross-chain attestor coordination.

8. Conclusion

Two paragraphs. (1) Cross-chain attestation portability is one shared mechanism, not five separate designs. The unified IBC light-client proof primitive plus freshness commitment handles all five upstream papers' extensions. Bounded staleness is the shared property; per-extension damage analysis follows from IBC update cadence. (2) The engineering choice (IBC vs Hyperlane vs restaking) is downstream of this paper; the paper specifies the primitive in IBC vocabulary but the mechanism generalizes. Roadmap is phased; v3+ ships the full surface once the chain-side engineering supports it.

References

[v0.1: References to fill in at v0.2. Anchors:]

1. PoUA paper (this repo, papers/poua/), arXiv:2605.25844.
 2. Native Delegation paper (this repo, papers/native-delegation/) §10.2.
 3. Per-Schema Fees paper (this repo, papers/per-schema-fees/) §9.4.
 4. Cross-Schema Composition paper (this repo, papers/cross-schema-composition/) §9.1.
 5. Time-Locked Attestations paper (this repo, papers/time-locked-attestations/) §9.2.
 6. Native DA Layer paper (this repo, papers/native-da/) §13.4.
 7. Cosmos IBC specification.
 8. Hyperlane technical documentation.
 9. EIP-7002 (Ethereum triggerable exits) specification.
 10. EIP-7251 (Ethereum max effective balance) specification.
 11. EigenLayer restaking specification.
-

Appendix A: Per-extension worked example

[v0.1: At v0.2: a worked example for the v1.5-first-extension recommendation (cross-chain delegation). Trace an Iris-relayed agent transaction on a Cosmos counterparty using a Ligate-side delegation grant: chain transitions on both sides, IBC packet contents, light-client verification steps, latency profile.]

Appendix B: IBC packet format spec

[v0.1: At v0.2: byte-level specification of the IBC packet format Ligate emits and consumes for each of the five extensions. Field layouts, signing, integrity checks.]